

MATA54 - Estruturas de Dados e Algoritmos II

Departamento de Ciência da Computação
Instituto de Computação
UFBA

inclui material de notas de aula dos professores
George Lima e Flávio Assis

Compressão de dados

Motivação: remoção da redundância na representação de informação

Aplicações

Qualquer aplicação que objetiva a economia de memória (armazenamento) e tempo (transmissão) de dados

Abordagens a serem estudadas: compressão sem perdas

- ▶ Métodos estatísticos
 - ▶ Huffman estático
 - ▶ Huffman dinâmico
- ▶ Métodos baseados em dicionário
 - ▶ Lempel-Ziv 77
 - ▶ Lempel-Ziv 78
 - ▶ LZW

Cocertos iniciais

Auto-informação

Quantidade de informação

Se A é um evento, quanto menos provável é sua ocorrência, maior a “quantidade” de informação a ele associada.

$$I(A) = \log_2 \frac{1}{\Pr(A)} = -\log_2 \Pr(A)$$

A auto-informação associada a uma sequência de eventos, $A_1 A_2 \dots A_n$ independentes é

$$\begin{aligned} I(A_1 A_2 \dots A_n) &= -\log_2 \Pr(A_1 A_2 \dots A_n) \\ &= -\log_2 \left(\prod_i^n A_i \right) = -\sum_1^n \log_2 A_i = \sum_1^n I(A_i) \end{aligned}$$

Entropia da informação

Auto-informação média

Entropia é a auto-informação média contida numa sequência de eventos $A_1 A_2 \dots A_n$. Assumindo independência...

$$H(A_1 A_2 \dots A_n) = \sum \Pr(A_i) I(A_i) = - \sum \Pr(A_i) \log_2 \Pr(A_i)$$

Claude Shannon (1948): nenhuma codificação de uma **sequência de eventos** pode ter tamanho menor que a sua entropia a menos que haja perda de informação.

- ▶ Evento $A_i \equiv$ **ocorrência de um símbolo** numa **mensagem**
- ▶ Mensagem $\equiv$ experimentos de retirar símbolos $A_i \in \Sigma$
- ▶ Geralmente, não é possível medir a entropia de uma mensagem.
Estimativas da entropia baseiam-se na **frequência** dos símbolos na mensagem

Modelo

Conjunto de regras que definem a ocorrência de símbolos e/ou formação da mensagem, que capturam um ou mais aspectos, do tipo:

- ▶ Frequência dos relativa símbolos
- ▶ Conhecimento sobre palavras (sequência de símbolos)
- ▶ Representação da dependência da ocorrência de símbolos ou palavras
- ▶ Adaptativo ou estático

Ex.: frequência das palavra mais usadas no português; frequência das letras num texto

Código e Codificação

Seja a sequência:

aaaabbcaaaddbcddeaaaabbbbbaa

Suponha a seguinte representação (hipotética) de 8 bits:

a: 01010001 b: 01110110 c: 10011101 d: 01010101 e: 01100011

A codificação da sequência seria:

<u>01010001</u>	<u>01010001</u>	<u>01010001</u>	<u>01010001</u>	<u>01110110</u>	<u>01110110</u>	<u>10011101</u>
^a	^a	^a	^a	^b	^b	^c
<u>01010001</u>	<u>01010001</u>	<u>01010001</u>	<u>01010101</u>	<u>01010101</u>	<u>01110110</u>	<u>10011101</u>
^a	^a	^a	^d	^d	^b	^c
<u>01010101</u>	<u>01010101</u>	<u>01100011</u>	<u>01010001</u>	<u>01010001</u>	<u>01010001</u>	<u>01110110</u>
^d	^d	^e	^a	^a	^a	^b
<u>01110110</u>	<u>01110110</u>	<u>01110110</u>	<u>01010001</u>	<u>01010001</u>		
^b	^b	^b	^a	^a		

208 bits! É possível representar os dados com menos bits?

Código e Codificação

Seja a sequência:

aaaabbcaaaddbbcddeaaaabbbbbaa

Poderíamos considerar a frequência de ocorrência dos caracteres:

a: 12 vezes b: 7 vezes c: 2 vezes d: 4 vezes e: 1 vez

Caracteres mais frequentes poderiam ter representações menores.

Exemplo:

a: 0 b: 1 c: 00 d: 01 e: 010

A codificação da sequência seria:

0	0	0	0	1	1	00	0	0	0	01	01	1	00
<i>a</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>b</i>	<i>b</i>	<i>c</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>d</i>	<i>d</i>	<i>b</i>	<i>c</i>
01	01	010	0	0	0	1	1	1	1	0	0		
<i>d</i>	<i>d</i>	<i>e</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>b</i>	<i>b</i>	<i>b</i>	<i>b</i>	<i>a</i>	<i>a</i>		

34 bits! Mas há um problema!

Código e Codificação

Seja a sequência:

aaaabbcaaaddbcddeaaabbbbaa

Poderíamos considerar a frequência de ocorrência dos caracteres:

a: 12 vezes b: 7 vezes c: 2 vezes d: 4 vezes e: 1 vez

Caracteres mais frequentes poderiam ter representações menores.

Exemplo:

a: 0 b: 1 c: 00 d: 01 e: 010

A codificação da sequência seria:

0	0	0	0	1	1	00	0	0	0	01	01	1	00
01	01	010	0	0	0	1	1	1	1	0	0		

34 bits! Mas há um problema! **Redundância da decodificação**

O que fazer? **Nenhuma representação pode ser prefixo de outra!**

Código e Codificação

Seja a sequência:

aaaabbcaaaaddbcbdeaaaabbbbaa

Codificação livre de prefixos:

a: 1 b: 01 c: 001 d: 0000 e: 0001

A sequência ficaria:

$\underbrace{1}_a \underbrace{1}_a \underbrace{1}_a \underbrace{1}_a \underbrace{01}_b \underbrace{01}_b \underbrace{001}_c \underbrace{1}_a \underbrace{1}_a \underbrace{1}_a \underbrace{0000}_d \underbrace{0000}_d \underbrace{01}_b \underbrace{001}_c$
 $\underbrace{0000}_d \underbrace{0000}_d \underbrace{0001}_e \underbrace{1}_a \underbrace{1}_a \underbrace{1}_a \underbrace{01}_b \underbrace{01}_b \underbrace{01}_b \underbrace{01}_b \underbrace{1}_a \underbrace{1}_a$

52 bits! Livre de redundância na decodificação!

Existe codificação livre de prefixos melhor que essa para esta sequência?

Código e codificação

Atribuição de sequência de bits à símbolos ou sequência de símbolos.

- ▶ Mínima redundância (**desejável**)
- ▶ Tamanho fixo ou variável (variável é **preferível**)
- ▶ Unicamente decodificável (**mandatório**)
- ▶ Decodificação eficiente (**muito importante**)

Ex.: Mensagem: *aaabbcbbaaabbcc*

- ▶ $a : 1; b : 00; c : 01$
- ▶ $a : 0; b : 11; c : 10$
- ▶ $aaa : 0; bb : 11; c : 10$

Métodos estadísticos

Huffman (estático): codificação eficiente por símbolo

Código de prefixo e otimalidade

- ▶ Nenhum código atribuído a um símbolo é prefixo de outros
- ▶ Codificação de prefixo com tamanho médio mínimo
 - ▶ Símbolos que ocorrem com menor frequência possuem maior código que aqueles que ocorrem com maior frequência
 - ▶ Os dois menos frequentes símbolos tem código de mesmo tamanho
- ▶ Código de Huffman (estático) é um código de prefixo ótimo

Código de Huffman: geração

Código de Huffman (1952)

Ideia básica: gerar uma árvore binária ponderada

1. Seja L uma lista dos símbolos ordenada por frequência
2. Retire os dois símbolos s_i e s_j de menor frequência f_i e f_j
3. Gere um novo símbolo s com frequência $f = f_i + f_j$ e inclua-o em L
4. Gere um nó numa árvore binária que representa s com filhos s_i e s_j
5. Retorne a (2) até que só haja um símbolo (raíz da árvore)
6. Gere um código para cada símbolo s_i representando o caminho da raíz até s_i

Huffman: Ilustração

Codificação de Huffman

Sequência: **abbdacccaabadaeeaa**

Frequências:

a: 8/18 b: 3/18 c: 3/18 d: 2/18 e: 2/18

Menores frequências: e e d

Nova lista:

a: 8/18 b: 3/18 c: 3/18 de: 4/18



Huffman: Ilustração

Codificação de Huffman

Sequência: **abbdacccaabadaeeaa**

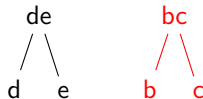
Lista:

a: 8/18 b: 3/18 c: 3/18 de: 4/18

Menores frequências: b e c

Nova lista:

a: 8/18 bc: 6/18 de: 4/18



Huffman: Ilustração

Codificação de Huffman

Sequência: **abbdaccaabadaeeaa**

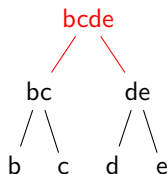
Lista:

a: 8/18 bc: 6/18 de: 4/18

Menores frequências: bc e de

Nova lista:

a: 8/18 bcde: 10/18



Huffman: Ilustração

Codificação de Huffman

Sequência: **abbdacccaabadaeeaa**

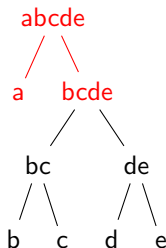
Lista:

a: 8/18 bcde: 10/18

Menores frequências: a e bcde

Nova lista:

abcde: 18/18

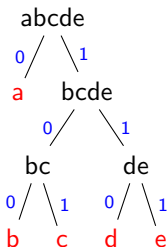


Huffman: Ilustração

Codificação de Huffman

Sequência: **abbdaccaabadaeeaa**

Geração da codificação: atribuição de 0 e 1 às arestas



Codificação: **a: 0** **b: 100** **c: 101** **d: 110** **e: 111**

Cod. da sequência: **38 bits!**

0 100 100 110 0 101 101 101 0 0 100 0 110 0 111 111 0 0
a b b d a c c c a a b a d a e e a a

Otimidade do código de Huffman

Árvore de custo mínimo

Seja T uma árvore a partir da qual se produz um **código ótimo** para uma mensagem de n símbolos.

$$w(T) = \sum_{i=1}^n f_i c_i$$

Sejam s_i e s_j dois símbolos de **frequência mínima** com códigos de tamanho c_i e c_j , respectivamente. Se c_i e c_j não são os maiores códigos, então selecione c_l e c_k de tamanho máximo e troque as folhas s_k e s_l de T por s_i e s_j , formando uma nova árvore T' . Como

$$w(T') \leq w(T)$$

Então, T pode não ser a melhor árvore!

Otimidade do código de Huffman

Árvore de Huffman é de custo mínimo

Informalmente...

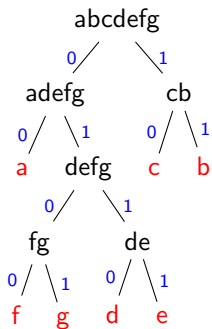
- ▶ A cada passo do algoritmo, os dois símbolos de menor frequência, s_i e s_j , são selecionados. A árvore para estes dois símbolos é de custo mínimo, pois estes são folhas cuja distância para raiz não pode ser maior que aquelas geradas para os demais símbolos.
- ▶ Os dois símbolos s_i e s_j são removidos do alfabeto e um novo símbolo é nele inserido, com frequência $f_i + f_j$.
- ▶ Repetindo os dois argumentos anteriores, para o novo conjunto de símbolos, pode-se concluir que a árvore gerada é de custo mínimo.

Exercício

Apresente uma decodificação de Huffman para a sequência:

dabeeababbebeaeadcccaaddcbbaccaaccaabccffbfbfdg

Resposta (uma das alternativas):



Codificação:

a: 00 b: 11 c: 10 d: 0110
e: 0111 f: 0100 g: 0101

Exercício (Cont. da Resposta)

Codificação da sequência:

dabeeababbebeaeadcccaaddcbbaccaaccaabccffbfbfbdg

0110	00	11	0111	0111	00	11	00	11	11	0111	11	0111	00	0111	00	0110
d	a	b	e	e	a	b	a	b	b	e	b	e	a	e	a	d
10	10	10	00	00	0110	0110	10	11	11	00	10	10	00	00	10	10
c	c	c	a	a	d	d	c	b	b	a	c	c	a	a	c	c
00	00	11	10	10	0100	0100	11	0100	11	0100	11	0110	0101			
a	a	b	c	c	f	f	b	f	b	f	b	d	g			

126 bits + bits para representar a árvore e os símbolos

Descompressão usando o código de Huffman

Como passar a árvore para o descompressor?

Observações

- ▶ A árvore de **Huffman é completa**
- ▶ A árvore é o dicionário que mapeia os símbolos e os respectivos códigos
- ▶ Código é de tamanho variável

Qualquer esquema deve requisitar o **mínimo de espaço** possível. Possível solução:

- ▶ Encaminhamento em árvore binária para visitar todas as folhas numa ordem pré-determinada
- ▶ Como a árvore é completa, encaminhamento pode estar associado a um código que representa a ordem da visita

Exercício (Cont. da Resposta)

Cabeçalho: árvore e símbolos

0010011011011afgdecb****

- ▶ Sequência de bits obtida através de um encaminhamento em-ordem na árvore
- ▶ Bit 1 após a árvore quebra a sequência do encaminhamento (flag)
- ▶ Sequência de símbolos segue a mesma ordem de visitação das folhas
- ▶ Esquema funciona porque a árvore é completa

Huffman dinâmico

Modelo estático vs dinâmico (ou adaptativo)

- ▶ Otimalidade do código de Huffman (estático) é ancorado na premissa que o modelo (baseado em frequência) representa a mensagem
- ▶ Ocorrências de símbolos podem **ser dependentes do contexto** ao longo da mensagem
- ▶ Modelos dinâmicos capturam mudanças de contexto, adaptando-se à mensagem ao longo de sua codificação
- ▶ A árvore de Huffman pode ser modificada ao longo da codificação!
 - ▶ Codificador e decodificador trabalham **sincronamente**

Huffman dinâmico: ideia básica

Compressão: árvore é dinamicamente construída

Seja λ um símbolo especial, que não ocorre em nenhuma mensagem.

1. Construa uma árvore de Huffman inicial contendo λ (o único símbolo inicialmente presente na árvore)
2. Repita
 - 2.1 Ler um novo símbolo s
 - 2.2 Se s é um símbolo ainda não lido (não presente na árvore): escreva λs na saída
 - 2.3 Caso contrário: escreva o código de s segundo o estado corrente da árvore
 - 2.4 Atualizada para refletir a ocorrência de s
3. Até que s indique o final da mensagem

Huffman dinâmico: árvore enumerada e ordenada

Características da árvore

Observações:

- ▶ Uma árvore binária com n folhas possui $2n - 1$ nós no total
- ▶ Cada nó folha numa árvore de Huffman, corresponde a um símbolo (de Alfabeto $\Sigma \cup \{\lambda\}$)
- ▶ Cada nó da árvore (folha ou interno) está associado a um número x_i e um peso w_i
- ▶ Os $2n - 1$ nós são enumerados tal que (**sibling property**):
 - ▶ Seus respectivos pesos w_i estão em ordem,
 $w_i \leq w_{i+1}, i = 1, 2, \dots, 2n - 2$
 - ▶ Os nós adjacentes x_{2j-1} e x_{2j} são filhos do mesmo pai, $1 \leq j < n$
 - ▶ O nó pai tem numeração maior que os filhos

Huffman dinâmico: códigos para símbolos do alfabeto

Economizando bits

Para $m = |\Sigma|$, escolhe-se e e r tal que $m = 2^e + r$ e $0 \leq r < 2^e$, de forma a associar um código único a cada símbolo (e de menor tamanho que o código usual)

- ▶ Para $\Sigma = \{s_1, s_2, \dots, s_m\}$, o código de s_k é
 - ▶ A representação binária de $k - 1$ com $e + 1$ bits se $1 \leq k \leq 2^e$
 - ▶ A representação binária de $k - r - 1$ com e bits.
- ▶ Para $\Sigma = \{A, B, \dots, Z\}$, $m = 26$. Então, $e = 4$ e $r = 10$
 - ▶ $A : 00000, B : 00001, \dots, T : 10100$
 - ▶ $U : 1010, V : 1011, Z : 1111$
- ▶ Nesta apresentação, não nos preocuparemos com este detalhe; o próprio símbolo para representar sua codificação

Huffman dinâmico: codificação e decodificação

Árvore inicial

Apenas um nó representando λ com $w = 0$ está presente

- ▶ Para alfabeto com m símbolos, enumera-se os nós a partir de $2n - 1$
- ▶ Inicialmente, $(2m - 1, 0, \lambda)$ é o único nó

Exemplo: ABBAACADBBCBB

Entropia $H = 1.738149$ bits/símbolo.

- ▶ Codificar a mensagem com Huffman dinâmico supondo $m = |\Sigma| = 26$
- ▶ Comparar o código gerado por símbolo com H .
- ▶ Comparar o código gerado com o código de Huffman estático
- ▶ Como seria a decodificação?

Métodos baseados em dicionário

Motivação

Ao invés de símbolos, palavras

- ▶ Se houver um dicionário de palavras, códigos podem ser índices do dicionário
- ▶ Sequências de símbolos (palavras) podem capturar melhor a correlação entre símbolos
- ▶ Dicionário deve ser adaptativo ao texto sendo codificado

Principais métodos

- ▶ Lempel-Ziv 1977 (LZ77 ou LZ1): códigos gerados são 3-tuplas
- ▶ Lempel-Ziv 1978 (LZ78 ou LZ2): códigos gerados são 2-tuplas
- ▶ Lempel-Ziv-Welch (LZW): códigos gerados são índices (unidimensionais) – Melhoria sobre LZ78

Janela deslizante

O dicionário é uma porção do que já foi visto até então

- ▶ Uma janela (*buffer*) é dividida em duas partes, **de busca** (*search*) e **de reconhecimento** (*look-ahead buffer*)
 - ▶ de busca: contém o que já visto e codificado da mensagem
 - ▶ de reconhecimento: contém o que está para ser codificado
- ▶ O índice do dicionário (código gerado) é uma tripla $\langle o, l, c \rangle$, que toma como base uma palavra p que já foi reconhecida e está no início da janela de reconhecimento:
 - ▶ o : distância para o início de p na janela de busca
 - ▶ l : tamanho de p
 - ▶ c : caracter seguinte a p na janela de reconhecimento
- ▶ Após gerar o código referente a palavra p seguida de c , pc a janela de busca passa a conter pc

LZ77: Ilustração

Codificação

Exemplo: ABRACADABRACADACABRA

► $\langle 0, 0, A \rangle \langle 0, 0, B \rangle \langle 0, 0, R \rangle \langle 3, 1, C \rangle \langle 2, 1, D \rangle \langle 7, 8, C \rangle \langle 9, 3, A \rangle$

Decodificação

Exemplo: $\langle 0, 0, A \rangle \langle 0, 0, B \rangle \langle 2, 9, A \rangle$

► ABABABABABAA

LZ77: Características

- ▶ Dicionário adaptativo
 - ▶ Nenhum conhecimento prévio sobre a mensagem é necessário
- ▶ Assume implicitamente que palavras próximas estão correlacionadas
- ▶ Como tamanho da janela deslizante é finita, **parte do que já foi reconhecido é perdido**
- ▶ Uso de triplas para indexar o dicionário pode gerar redundância quando não há repetições de longas palavras
- ▶ Índice pode ser usando codificação de tamanho variável (ex.: Huffman)
- ▶ Vários compressores conhecidos usam alguma versão de LZ77
 - ▶ PKZip, Zip, LHArc, PNG, gzip, ARJ

Dicionário sem limite

O dicionário é o que já foi visto até então

- ▶ Cada palavra nova é incluída no dicionário, fazendo referências a entradas anteriores
- ▶ O índice do dicionário (código gerado) para uma palavra pc é uma tupla $\langle i, c \rangle$
 - ▶ i : índice da palavra já p contida no dicionário
 - ▶ c : caracter seguinte a p
- ▶ Após gerar o código referente à palavra p seguida de c , a palavra pc é inserida no dicionário

LZ78: Ilustração

Codificação

Exemplo: ABRACADABRACADACABRA

- ▶ $\langle 0, A \rangle \langle 0, B \rangle \langle 0, R \rangle \langle 1, C \rangle \langle 1, D \rangle \langle 1, B \rangle \langle 3, A \rangle \langle 0, C \rangle \langle 5, A \rangle \langle 8, A \rangle$
 $\langle 2, R \rangle \langle 0, A \rangle$

Decodificação

Exemplo: $\langle 0, A \rangle \langle 0, B \rangle \langle 1, B \rangle \langle 3, A \rangle \langle 2, A \rangle \langle 5, A \rangle$

- ▶ ABABABABABAA

LZ78: Características

- ▶ Dicionário adaptativo
 - ▶ Nenhum conhecimento prévio sobre a mensagem é necessário
- ▶ Dicionário tem potencial de crescer indefinidamente
 - ▶ Nada se perde
 - ▶ Taxa de compressão pode ficar comprometida se o dicionário cresce demasiadamente
 - ▶ Representação do índice é função do tamanho do dicionário
 - ▶ Esforço para codificar/decodificar aumenta
- ▶ Pode-se monitorar o tamanho do dicionário e a taxa de compressão, reiniciando o dicionário a partir de determinado momento
- ▶ As implementações de LZ78 são geralmente baseadas em LZW, pois este fornece um melhoramento

Lempel-Ziv-Welch (LZW)

- ▶ Método **baseado em dicionário**: mantém registro sobre sequências de símbolos encontrados ao longo da entrada
- ▶ É **adaptativo**, pois define sequências de símbolos a serem codificadas à medida que se lê a entrada
- ▶ Não requer o armazenamento ou transmissão de tabelas de codificação - requer apenas o conhecimento da codificação de símbolos básicos, previamente conhecidos

Codificação

- ▶ Inicia-se com uma codificação para o alfabeto de símbolos básicos da entrada
- ▶ Adicionam-se ao dicionário sequências de símbolos (**palavras**) que vão sendo conhecidas

Codificação - Ilustração

Exemplo de entrada

aabaaacaade

a a b a a a c a a d e

↑

p:

a

Está no dicionário? Sim. Avança

Saída:

Dicionário

a: 0 **d:** 3

b: 1 **e:** 4

c: 2

Codificação - Ilustração

Exemplo de entrada

aabaaacaade

a a b a a a c a a d e

↑

p: **aa** Está no dicionário? Não!

Saída:

Dicionário

a: 0 **d:** 3

b: 1 **e:** 4

c: 2

Codificação - Ilustração

Exemplo de entrada

aabaaacaade

a a b a a a c a a d e
 ↑
p: **aa** Está no dicionário? Não!

Saída:

Insere-se **p** na próxima posição do dicionário; insere-se na saída a palavra de código relativo a **p** sem o último símbolo (**a**); e **p** passa a conter apenas o último símbolo.

Dicionário

a: 0 **d:** 3

b: 1 **e:** 4

c: 2

Codificação - Ilustração

Exemplo de entrada

aabaaacaade

a a b a a a c a a d e



p: a

Saída: 0

Dicionário

a: 0 **d:** 3

b: 1 **e:** 4

c: 2 **aa:** 5

Codificação - Ilustração

Exemplo de entrada

aabaaacaade

a a b a a a c a a d e
 ↑
p: **ab** Está no dicionário? Não!

Saída: 0

Insere-se **p** na próxima posição do dicionário; insere-se na saída a palavra de código relativo a **p** sem o último símbolo (**a**); e **p** passa a conter apenas o último símbolo.

Dicionário

a: 0 **d:** 3

b: 1 **e:** 4

c: 2 **aa:** 5

Codificação - Ilustração

Exemplo de entrada

aabaaacaade

a a b a a a c a a d e



p: b

Saída: 0 0

Dicionário

a: 0 d: 3 ab: 6

b: 1 e: 4

c: 2 aa: 5

Codificação - Ilustração

Exemplo de entrada

aabaaacaade

a a b a a a c a a d e



p: **ba** Está no dicionário? Não!

Saída: 0 0

Insere-se **p** na próxima posição do dicionário; insere-se na saída a palavra de código relativo a **p** sem o último símbolo (**b**); e **p** passa a conter apenas o último símbolo.

Dicionário

a: 0 **d:** 3 **ab:** 6

b: 1 **e:** 4

c: 2 **aa:** 5

Codificação - Ilustração

Exemplo de entrada

aabaaacaade

a a b a a a c a a d e



p: a

Saída: 0 0 1

Dicionário

a: 0	d: 3	ab: 6
b: 1	e: 4	ba: 7
c: 2	aa: 5	

Codificação - Ilustração

Exemplo de entrada

aabaaacaade

a a b a a a c a a d e



p: **aa** Está no dicionário? Sim! Avança

Saída: 0 0 1

Dicionário

a: 0	d: 3	ab: 6
b: 1	e: 4	ba: 7
c: 2	aa: 5	

Codificação - Ilustração

Exemplo de entrada

aabaaacaade

a a b a a a c a a d e



p: **aaa** Está no dicionário? Não!

Saída: 0 0 1

Insere-se **p** na próxima posição do dicionário; insere-se na saída a palavra de código relativo a **p** sem o último símbolo (**aa**); e **p** passa a conter apenas o último símbolo.

Dicionário

a: 0	d: 3	ab: 6
b: 1	e: 4	ba: 7
c: 2	aa: 5	

Codificação - Ilustração

Exemplo de entrada

aabaaacaade

a a b a a a c a a d e



p: a

Saída: 0 0 1 5

Dicionário

a: 0	d: 3	ab: 6
b: 1	e: 4	ba: 7
c: 2	aa: 5	aaa: 8

Codificação - Ilustração

Exemplo de entrada

aabaaacaade

a a b a a a c a a d e



p: **ac** Está no dicionário? Não!

Saída: 0 0 1 5

Insere-se **p** na próxima posição do dicionário; insere-se na saída a palavra de código relativo a **p** sem o último símbolo (**a**); e **p** passa a conter apenas o último símbolo.

Dicionário

a: 0 **d:** 3 **ab:** 6

b: 1 **e:** 4 **ba:** 7

c: 2 **aa:** 5 **aaa:** 8

Codificação - Ilustração

Exemplo de entrada

aabaaacaade

a a b a a a c a a d e



p: c

Saída: 0 0 1 5 0

Dicionário

a: 0	d: 3	ab: 6	ac: 9
b: 1	e: 4	ba: 7	
c: 2	aa: 5	aaa: 8	

Codificação - Ilustração

Exemplo de entrada

aabaaacaade

a a b a a a c a a d e

p: **ca** Está no dicionário? Não!

Saída: 0 0 1 5 0

Insere-se **p** na próxima posição do dicionário; insere-se na saída a palavra de código relativo a **p** sem o último símbolo (**c**); e **p** passa a conter apenas o último símbolo.

Dicionário

a: 0	d: 3	ab: 6	ac: 9
b: 1	e: 4	ba: 7	
c: 2	aa: 5	aaa: 8	

Codificação - Ilustração

Exemplo de entrada

aabaaacaade

a a b a a a c a a d e



p: a

Saída: 0 0 1 5 0 2

Dicionário

a: 0	d: 3	ab: 6	ac: 9
b: 1	e: 4	ba: 7	ca: 10
c: 2	aa: 5	aaa: 8	

Codificação - Ilustração

Exemplo de entrada

aabaaacaade

a a b a a a c a a d e

p: **aa** Está no dicionário? **Sim!** Avança
↑

Saída: 0 0 1 5 0 2

Dicionário

a: 0	d: 3	ab: 6	ac: 9
b: 1	e: 4	ba: 7	ca: 10
c: 2	aa: 5	aaa: 8	

Codificação - Ilustração

Exemplo de entrada

aabaaacaade

a a b a a a c a a d e

p: **aad** Está no dicionário? Não!

Saída: 0 0 1 5 0 2

Insere-se **p** na próxima posição do dicionário; insere-se na saída a palavra de código relativo a **p** sem o último símbolo (**aa**); e **p** passa a conter apenas o último símbolo.

Dicionário

a: 0	d: 3	ab: 6	ac: 9
b: 1	e: 4	ba: 7	ca: 10
c: 2	aa: 5	aaa: 8	

Codificação - Ilustração

Exemplo de entrada

aabaaacaade

a a b a a a c a a d e



p: d

Saída: 0 0 1 5 0 2 5

Dicionário

a: 0	d: 3	ab: 6	ac: 9
b: 1	e: 4	ba: 7	ca: 10
c: 2	aa: 5	aaa: 8	aad: 11

Codificação - Ilustração

Exemplo de entrada

aabaaacaade

a a b a a a c a a d e



p: **de** Está no dicionário? Não!

Saída: 0 0 1 5 0 2 5

Insere-se **p** na próxima posição do dicionário; insere-se na saída a palavra de código relativo a **p** sem o último símbolo (**d**); e **p** passa a conter apenas o último símbolo.

Dicionário

a: 0	d: 3	ab: 6	ac: 9
b: 1	e: 4	ba: 7	ca: 10
c: 2	aa: 5	aaa: 8	aad: 11

Codificação - Ilustração

Exemplo de entrada

aabaaacaade

a a b a a a c a a d e
↑

p: e

Saída: 0 0 1 5 0 2 5 3

Dicionário

a: 0	d: 3	ab: 6	ac: 9	de: 12
b: 1	e: 4	ba: 7	ca: 10	
c: 2	aa: 5	aaa: 8	aad: 11	

Codificação - Ilustração

Exemplo de entrada

aabaaacaade

a a b a a a c a a d e
↑

p: **e** Está no dicionário? Sim!

Saída: 0 0 1 5 0 2 5 3 4

Como terminou a entrada, insere o código de **p** na saída.

Dicionário

a: 0	d: 3	ab: 6	ac: 9	de: 12
b: 1	e: 4	ba: 7	ca: 10	
c: 2	aa: 5	aaa: 8	aad: 11	

Codificação LZW

Algoritmo 1: Compressão LZW

entrada : Texto T composto de uma sequência de símbolos de um alfabeto

$$\Sigma = \{s_0, s_1, \dots, s_{|\Sigma|-1}\}$$

saida : Sequência de inteiros representando o código gerado para T

```
1  $D \leftarrow \{(k, s_k) | s_k \in \Sigma, k = 0, 1, \dots, |\Sigma| - 1\}$ ;           // Cria dicionário  $D$ 
2  $k \leftarrow |D|$ ;
3  $p \leftarrow ""$ ;                                           // palavra a ser buscada no dicionário
4 foreach  $s \in T$  do
5     if  $p \cdot s \in D$  then
6          $p \leftarrow p \cdot s$ ;                               // concatena  $p$  com  $s$ 
7     else
8          $\text{output}(D.\text{index}(p))$ ;           //  $D.\text{index}(p)$  = código de  $p$  em  $D$ 
9          $D \leftarrow D \cup \{(k, p \cdot s)\}$ ;           // Atualiza  $D$ 
10         $k \leftarrow k + 1$ ;
11         $p \leftarrow s$ ;
12  $\text{output}(D.\text{index}(p))$ ;
```

Decodificação - Ilustração

Na decodificação, segue-se o **mesmo princípio** da codificação! A partir dos valores da codificação, vai-se gerando a saída e repetindo o processo usado na codificação para montar o dicionário:

	0	0	1	5	0	2	5	3	4
	↑								
Saída:	a								
	↑								
p:	a								

0 é **a**. Portanto, insere-se o **a** na saída. Há **p** no dicionário? Sim, então segue-se a decodificação

Dicionário

a: 0 **d:** 3
b: 1 **e:** 4
c: 2

Decodificação - Ilustração

Na decodificação, segue-se o **mesmo princípio** da codificação! A partir dos valores da codificação, vai-se gerando a saída e repetindo o processo usado na codificação para montar o dicionário:

	0	0	1	5	0	2	5	3	4
		↑							
Saída:	a	a							
		↑							
p:	aa								

0 é novamente **a**. Concatena-o em **p**. Há **p** no dicionário? Não! Então, insere-se **p** na próxima posição do dicionário e **p** fica apenas com o último símbolo

Dicionário

a: 0 d: 3
b: 1 e: 4
c: 2

Decodificação - Ilustração

Na decodificação, segue-se o **mesmo princípio** da codificação! A partir dos valores da codificação, vai-se gerando a saída e repetindo o processo usado na codificação para montar o dicionário:

	0	0	1	5	0	2	5	3	4
		↑							
Saída:	a	a							
		↑							
p:	a								

Dicionário

a: 0 **d:** 3

b: 1 **e:** 4

c: 2 **aa:** 5

Decodificação - Ilustração

Na decodificação, segue-se o **mesmo princípio** da codificação! A partir dos valores da codificação, vai-se gerando a saída e repetindo o processo usado na codificação para montar o dicionário:

	0	0	1	5	0	2	5	3	4
			↑						
Saída:	a	a	b						
			↑						
p:		ab							

1 é **b**. Coloca-o na saída. Segue-e com **p**. Há **p** no dicionário? Não! Então, insere-se **p** na próxima posição do dicionário e **p** fica apenas com o último símbolo

Dicionário

a: 0 **d:** 3

b: 1 **e:** 4

c: 2 **aa:** 5

Decodificação - Ilustração

Na decodificação, segue-se o **mesmo princípio** da codificação! A partir dos valores da codificação, vai-se gerando a saída e repetindo o processo usado na codificação para montar o dicionário:

	0	0	1	5	0	2	5	3	4
			↑						
Saída:	a	a	b						
			↑						
p:	b								

Dicionário

a: 0	d: 3	ab: 6
b: 1	e: 4	
c: 2	aa: 5	

Decodificação - Ilustração

Na decodificação, segue-se o **mesmo princípio** da codificação! A partir dos valores da codificação, vai-se gerando a saída e repetindo o processo usado na codificação para montar o dicionário:

	0	0	1	5	0	2	5	3	4
				↑					
Saída:	a	a	b	a	a				
				↑					
p:				ba					

5 é **aa**. Coloca-os na saída. Segue-e com **p**. Há **p** no dicionário? Não! Então, insere-se **p** na próxima posição do dicionário e **p** fica apenas com o último símbolo

Dicionário

a: 0	d: 3	ab: 6
b: 1	e: 4	
c: 2	aa: 5	

Decodificação - Ilustração

Na decodificação, segue-se o **mesmo princípio** da codificação! A partir dos valores da codificação, vai-se gerando a saída e repetindo o processo usado na codificação para montar o dicionário:

	0	0	1	5	0	2	5	3	4
				↑					
Saída:	a	a	b	a	a				
				↑					
p:	a								

Dicionário

a: 0	d: 3	ab: 6
b: 1	e: 4	ba: 7
c: 2	aa: 5	

Decodificação - Ilustração

Na decodificação, segue-se o **mesmo princípio** da codificação! A partir dos valores da codificação, vai-se gerando a saída e repetindo o processo usado na codificação para montar o dicionário:

	0	0	1	5	0	2	5	3	4
				↑					
Saída:	a	a	b	a	a				
					↑				
p:	aa								

Avança-se com **p**. Há **p** no dicionário? Sim!

Dicionário

a: 0	d: 3	ab: 6
b: 1	e: 4	ba: 7
c: 2	aa: 5	

Decodificação - Ilustração

Na decodificação, segue-se o **mesmo princípio** da codificação! A partir dos valores da codificação, vai-se gerando a saída e repetindo o processo usado na codificação para montar o dicionário:

	0	0	1	5	0	2	5	3	4
				↑					
Saída:	a	a	b	a	a	a			
					↑				
p:	aaa								

0 é **a**. Coloca-o na saída. Segue-e com **p**. Há **p** no dicionário? Não! Então, insere-se **p** na próxima posição do dicionário e **p** fica apenas com o último símbolo

Dicionário

a: 0	d: 3	ab: 6
b: 1	e: 4	ba: 7
c: 2	aa: 5	

Decodificação - Ilustração

Na decodificação, segue-se o **mesmo princípio** da codificação! A partir dos valores da codificação, vai-se gerando a saída e repetindo o processo usado na codificação para montar o dicionário:

	0	0	1	5	0	2	5	3	4
					↑				
Saída:	a	a	b	a	a	a			
						↑			
p:	a								

Dicionário

a: 0	d: 3	ab: 6
b: 1	e: 4	ba: 7
c: 2	aa: 5	aaa: 8

Decodificação - Ilustração

Na decodificação, segue-se o **mesmo princípio** da codificação! A partir dos valores da codificação, vai-se gerando a saída e repetindo o processo usado na codificação para montar o dicionário:

	0	0	1	5	0	2	5	3	4
						↑			
Saída:	a	a	b	a	a	a	c		
							↑		
p:							ac		

2 é **c**. Coloca-o na saída. Segue-e com **p**. Há **p** no dicionário? Não! Então, insere-se **p** na próxima posição do dicionário e **p** fica apenas com o último símbolo

Dicionário

a: 0	d: 3	ab: 6
b: 1	e: 4	ba: 7
c: 2	aa: 5	aaa: 8

Decodificação - Ilustração

Na decodificação, segue-se o **mesmo princípio** da codificação! A partir dos valores da codificação, vai-se gerando a saída e repetindo o processo usado na codificação para montar o dicionário:

	0	0	1	5	0	2	5	3	4
						↑			
Saída:	a	a	b	a	a	a	c		
							↑		
p:	c								

Dicionário

a: 0	d: 3	ab: 6	ac: 9
b: 1	e: 4	ba: 7	
c: 2	aa: 5	aaa: 8	

Decodificação - Ilustração

Na decodificação, segue-se o **mesmo princípio** da codificação! A partir dos valores da codificação, vai-se gerando a saída e repetindo o processo usado na codificação para montar o dicionário:

	0	0	1	5	0	2	5	3	4
							↑		
Saída:	a	a	b	a	a	a	c	a	a
								↑	
p:								ca	

5 é **aa**. Coloca-os na saída. Segue-e com **p**. Há **p** no dicionário? Não! Então, insere-se **p** na próxima posição do dicionário e **p** fica apenas com o último símbolo

Dicionário

a: 0	d: 3	ab: 6	ac: 9
b: 1	e: 4	ba: 7	
c: 2	aa: 5	aaa: 8	

Decodificação - Ilustração

Na decodificação, segue-se o **mesmo princípio** da codificação! A partir dos valores da codificação, vai-se gerando a saída e repetindo o processo usado na codificação para montar o dicionário:

	0	0	1	5	0	2	5	3	4
							↑		
Saída:	a	a	b	a	a	a	c	a	a
								↑	
p:	a								

Dicionário

a: 0	d: 3	ab: 6	ac: 9
b: 1	e: 4	ba: 7	ca: 10
c: 2	aa: 5	aaa: 8	

Decodificação - Ilustração

Na decodificação, segue-se o **mesmo princípio** da codificação! A partir dos valores da codificação, vai-se gerando a saída e repetindo o processo usado na codificação para montar o dicionário:

	0	0	1	5	0	2	5	3	4
							↑		
Saída:	a	a	b	a	a	a	c	a	a
								↑	
p:	aa								

Segue-se com **p**. Há **p** no dicionário? Sim!

Dicionário

a: 0	d: 3	ab: 6	ac: 9
b: 1	e: 4	ba: 7	ca: 10
c: 2	aa: 5	aaa: 8	

Decodificação - Ilustração

Na decodificação, segue-se o **mesmo princípio** da codificação! A partir dos valores da codificação, vai-se gerando a saída e repetindo o processo usado na codificação para montar o dicionário:

	0	0	1	5	0	2	5	3	4	
								↑		
Saída:	a	a	b	a	a	a	c	a	a	d
									↑	
p:	a	a	d							

3 é **d**. Coloca-o na saída. Segue-e com **p**. Há **p** no dicionário? Não! Então, insere-se **p** na próxima posição do dicionário e **p** fica apenas com o último símbolo

Dicionário

a: 0	d: 3	ab: 6	ac: 9
b: 1	e: 4	ba: 7	ca: 10
c: 2	aa: 5	aaa: 8	

Decodificação - Ilustração

Na decodificação, segue-se o **mesmo princípio** da codificação! A partir dos valores da codificação, vai-se gerando a saída e repetindo o processo usado na codificação para montar o dicionário:

	0	0	1	5	0	2	5	3	4	
								↑		
Saída:	a	a	b	a	a	a	c	a	a	d
									↑	
p:	d									

Dicionário

a: 0	d: 3	ab: 6	ac: 9
b: 1	e: 4	ba: 7	ca: 10
c: 2	aa: 5	aaa: 8	aad: 11

Decodificação - Ilustração

Na decodificação, segue-se o **mesmo princípio** da codificação! A partir dos valores da codificação, vai-se gerando a saída e repetindo o processo usado na codificação para montar o dicionário:

	0	0	1	5	0	2	5	3	4	
								↑		
Saída:	a	a	b	a	a	a	c	a	a	d e
									↑	
p:										d

4 é e. Coloca-o na saída. Termina-se a decodificação

Dicionário

a: 0	d: 3	ab: 6	ac: 9
b: 1	e: 4	ba: 7	ca: 10
c: 2	aa: 5	aaa: 8	aad: 11

Decodificação - Caso Especial

Suponha que se queira decodificar a sequência abaixo, considerando um alfabeto apenas com as letras **a** (**0**) e **b** (**1**)

	0	1	2	4	1
	↑				
Saída:	a				
	↑				
p:	a				

0 é **a**. Coloca-o na saída. Há **p** no dicionário? Sim!

Dicionário

a: 0

b: 1

Decodificação - Caso Especial

Suponha que se queira decodificar a sequência abaixo, considerando um alfabeto apenas com as letras **a** (**0**) e **b** (**1**)

	0	1	2	4	1
		↑			
Saída:	a	b			
		↑			
p:	ab				

1 é **b**. Coloca-o na saída. Há **p** no dicionário? Não! Então, insere-se **p** na próxima posição do dicionário e **p** fica apenas com o último símbolo

Dicionário

a: 0

b: 1

Decodificação - Caso Especial

Suponha que se queira decodificar a sequência abaixo, considerando um alfabeto apenas com as letras **a** (**0**) e **b** (**1**)

	0	1	2	4	1
		↑			
Saída:	a	b			
		↑			
p:	b				

Dicionário

a: 0

b: 1

ab: 2

Decodificação - Caso Especial

Suponha que se queira decodificar a sequência abaixo, considerando um alfabeto apenas com as letras **a** (**0**) e **b** (**1**)

	0	1	2	4	1
			↑		
Saída:	a	b	a	b	
			↑		
p:	ba				

2 é **ab**. Coloca-os na saída. Há **p** no dicionário? Não! Então, insere-se **p** na próxima posição do dicionário e **p** fica apenas com o último símbolo

Dicionário

a: 0

b: 1

ab: 2

Decodificação - Caso Especial

Suponha que se queira decodificar a sequência abaixo, considerando um alfabeto apenas com as letras **a** (**0**) e **b** (**1**)

	0	1	2	4	1
			↑		
Saída:	a	b	a	b	
			↑		
p:	a				

Dicionário

a: 0 **ba:** 3

b: 1

ab: 2

Decodificação - Caso Especial

Suponha que se queira decodificar a sequência abaixo, considerando um alfabeto apenas com as letras **a** (0) e **b** (1)

	0	1	2	4	1
			↑		
Saída:	a	b	a	b	
				↑	
p:	ab				

Segue-se com **p**. Há **p** no dicionário? Sim! Vamos prosseguir!

Dicionário

a: 0 **ba:** 3

b: 1

ab: 2

Decodificação - Caso Especial

Suponha que se queira decodificar a sequência abaixo, considerando um alfabeto apenas com as letras **a** (0) e **b** (1)

	0	1	2	4	1
			↑		
Saída:	a	b	a	b	
				↑	
p:	ab				

Ops! Não há 4 no dicionário! O que fazer?

Dicionário

a: 0 **ba:** 3

b: 1

ab: 2

Decodificação - Caso Especial

Suponha que se queira decodificar a sequência abaixo, considerando um alfabeto apenas com as letras **a** (0) e **b** (1)

	0	1	2	4	1
			↑		
Saída:	a	b	a	b	
				↑	
p:	ab				

Ops! Não há 4 no dicionário! O que fazer?

É possível saber qual é a entrada 4. Como?

Dicionário

a: 0 **ba:** 3

b: 1

ab: 2

Decodificação - Caso Especial

Suponha que se queira decodificar a sequência abaixo, considerando um alfabeto apenas com as letras **a** (0) e **b** (1)

	0	1	2	4	1
			↑		
Saída:	a	b	a	b	
				↑	
p:	ab				

A entrada 4 somente pode ser **aba**!

Dicionário

a: 0 **ba:** 3

b: 1

ab: 2

Decodificação - Caso Especial

Suponha que se queira decodificar a sequência abaixo, considerando um alfabeto apenas com as letras **a** (**0**) e **b** (**1**)

	0	1	2	4	1
			↑		
Saída:	a	b	a	b	
				↑	
p:	ab				

A entrada 4 somente pode ser **aba**!

Veja que, na codificação há o **2**, relativo a **ab**. Neste ponto, na codificação, será criada a entrada **4**.

Dicionário

a: 0 **ba:** 3

b: 1

ab: 2

Decodificação - Caso Especial

Suponha que se queira decodificar a sequência abaixo, considerando um alfabeto apenas com as letras **a** (0) e **b** (1)

	0	1	2	4	1
			↑		
Saída:	a	b	a	b	X
				↑	
p:	abX				

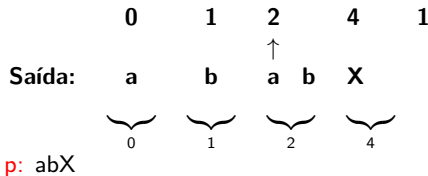
Como ela é criada? Ela será da forma **abX**, sendo **X** o próximo caractere.

Dicionário

a: 0	ba: 3
b: 1	abX: 4
ab: 2	

Decodificação - Caso Especial

Suponha que se queira decodificar a sequência abaixo, considerando um alfabeto apenas com as letras **a** (0) e **b** (1)



Mas qual caractere é **X**? Considerando que o próximo número da codificação é 4, a sequência que começa após o 2 é **abX**. Portanto **X** é **a**!

Dicionário

a: 0 **ba:** 3

b: 1 **abX:** 4

ab: 2

Decodificação - Caso Especial

Suponha que se queira decodificar a sequência abaixo, considerando um alfabeto apenas com as letras **a** (0) e **b** (1)

	0	1	2	4		1
			↑			
Saída:	a	b	a	b	X	
	a	b	a	b	a	b
	⏟		⏟		⏟	
	0		1		2	
p:	abX					

Mas qual caractere é **X**? Considerando que o próximo número da codificação é **4**, a sequência que começa após o **2** é **abX**. Portanto **X** é **a**!

Dicionário

a: 0 **ba:** 3

b: 1 **abX:** 4

ab: 2

Decodificação - Caso Especial

Suponha que se queira decodificar a sequência abaixo, considerando um alfabeto apenas com as letras **a** (0) e **b** (1)

Saída: $\underbrace{\quad}_0$ $\underbrace{\quad}_1$ $\underbrace{\quad}_2$ $\underbrace{\quad}_4$ $\underbrace{\quad}_1$
 a **b** **a** **b** **a** **b** **a** **b**

Com isso, seguindo-se o algoritmo, chega-se à decodificação final

Dicionário

a: 0 **ba:** 3

b: 1 **aba:** 4

ab: 2

Decodificação LZW

Algoritmo 2: Descompressão LZW

entrada : Código C gerado pelo compressor relacionado a um texto T com símbolos de um alfabeto $\Sigma = \{s_0, s_1, \dots, s_{|\Sigma|-1}\}$

saida : Sequência de símbolos que compõem o texto T

```
1  $D \leftarrow \{(k, s_k) | s_k \in \Sigma, k = 0, 1, \dots, |\Sigma| - 1\}$ ;           // Cria dicionário  $D$ 
2  $k \leftarrow |D|$ ;
3  $p \leftarrow ""$ ;                                     // palavra para compor o dicionário
4 foreach  $c \in C$  do
5     while  $c \notin D$  do
6          $j \leftarrow 1$ ;
7         while  $j \leq |p| \wedge p[:j] \in D$  do  $j \leftarrow j + 1$ ;
8         if  $j > |p|$  then                                     // Se  $p \in D$ 
9              $D \leftarrow D \cup \{(k, p \cdot p[0])\}$ ;           // Atualiza  $D$  (caso especial)
10             $p \leftarrow ""$ ;
11        else
12             $D \leftarrow D \cup \{(k, p[:j])\}$ ;                 // Atualiza  $D$  (caso padrão)
13             $p \leftarrow p[j - 1 :]$ ;
14         $k \leftarrow k + 1$ ;
15     $p \leftarrow p \cdot D[c]$ ;                                     // concatena  $p$  com  $D$  de índice  $c$ 
16    output( $D[c]$ );
```


Exercício de fixação

Considere o código gerado pelo compressor LZW abaixo, cujo dicionário inicial é composto dos símbolos 'a', 'b' e 'c', com respectivos códigos 0, 1 e 2. Encontre o texto original.

0 1 3 2 4 7 0 9 10 0